Technical University of Crete
School of Electrical and Computer Engineering

Big Data Processing and Analysis Project Report

Synopses and Full Dataset Utilization Comparison in Chicago Spatiotemporal Crime Count Predictor

Full Name: Koutsovasilis Vasileios
Registration Number: 2020030137

Introduction

This project utilizes Apache Spark's PySpark to analyze the reported crimes dataset in the city of Chicago and create a spatiotemporal crime count predictor using machine learning algorithms. The primary objective of this project is to examine the effect of using the full dataset, versus synopsis, of various sizes, to train the model and highlight the advantages and tradeoffs of each method, in the context of Big Data Analytics and traditional Regression models.

The Dataset

The reported Crimes Dataset reflects reported incidents of criminal activity that occurred in the city of Chicago from 2001 to 15/04/2025. The Dataset is provided by the police department of Chicago and is available for public use. It consists of **8.29 million** rows, **22** distinct columns and has a total size of **1.8 GB**. The description of each column is given below:

Column	Column Description
ID	Unique Identifier for the record.
Case Number	The CPD Records Division Number.
Date	Date when the incident was reported.
Block	The partially redacted address where the incident occurred.
IUCR	The Illinois Uniform Crime Reporting code.
Primary Type	The primary description of the IUCR code.
Description	Secondary description of the IUCR code, subcategory of primary type.
Location Description	Description of the location of the incident.
Arrest	Indicates whether an arrest was made.
Domestic	Indicates whether the incident was domestic related.
Beat	Indicates the beat where the incident occurred. A beat is the smallest geographic area with a dedicated police car.
District	The police district of the incident.
Ward	The ward (City Council district) of the incident.

Community Area	Indicates the Community Area where the incident occurred. Chicago has a total of 77 community areas.
FBI Code	Indicates the crime classification as outlined in the FBI's National Incident Based Reporting System (NIBRS).
X Coordinate	The partially redacted x coordinate of the location of the incident in State Plane Illinois East NAD 1983 projection.
Y Coordinate	The partially redacted y coordinate of the location of the incident in State Plane Illinois East NAD 1983 projection.
Year	Year the incident occurred
Updated On	Date and time the record was last updated.
Latitude	The partially redacted latitude of the location of the incident.
Longitude	The partially redacted longitude of the location of the incident.
Location	The partially redacted location of the incident in a format that allows for creation of maps and other geographic operations.

Methodology

The methodology for this project involves setting up the necessary environment, loading and preprocessing the dataset, creating the necessary synopses, implementing three distinct regression algorithms on the full dataset and the synopses and evaluating the results. Each of the following subsections highlights the steps taken in this study, in more detail.

- ## Spark Setup

  A PySpark session was set up in Google Collab along with Ngrok, to tunnel the IP address, enabling GUI visualization to better monitor Spark's progress.

- ## Importing the Dataset

  The Dataset for this project was downloaded from the City of Chicago Data Portal and uploaded to google Drive. Afterwards, it was imported to a Spark DataFrame and the proper schema was applied to the data, along with some transformations for the Dates to have the proper format (MM/dd/yy hh:mm:ss).

- ## Synopsis Creation Technique

  All of the synopses that will be used were created using the simple technique of reservoir sampling, as shown in the lectures of the course. To create the samples a generator function was defined to iterate over the data in each partition and return a sample. The synopsis creation technique is made to be able to handle distributed data without having to collect all of them in memory, which could be problematic due to their volume.

```python
def reservoir_sampling(iterator, sample_size):
    """
    Method to use to create a reservoir sample on each partition.
    """
    reservoir = []
    for i, value in enumerate(iterator):
        if i < sample_size:
            reservoir.append(value)
        else:
            random_num = random.randint(0, i)
            if random_num < sample_size:
                reservoir[random_num] = value

    yield reservoir
```

Figure 1: Method used to create the reservoir sample

The steps taken for the creation of the synopsis are the following:

1. Count the total rows in the dataset and calculate the rows the synopsis should have.
2. Cast date columns to string to preserve the date data and transform the DataFrame to an RDD.
3. Get the number of partitions for the RDD and calculate number of samples that should be taken from each partition, as well as the number of partitions that the final synopsis should be repartitioned into.
4. Create the synopsis RDD by mapping each partition using the reservoir sample generator function in two stages. In the first stage the `sample_size` parameter of the method is the samples per partition. This way we ensure that each partition will contribute a roughly even sample size and avoid skewing. Additionally, to avoid falling sort of the desired total rows the final synopsis should have, we can apply an oversampling rate, to increase the per partition sample size, to counter the loss due to uneven distribution of data or partition sizes. In the second stage the `sample_size` parameter takes on the value of the desired total sample rows that the synopsis should have. This way we further condense the data to match the desired size. After each stage a flatMap is applied as the result of the reservoir sample is an RDD of lists, so by flattening the list we can get the samples we wanted.
5. Transform the synopsis RDD into a Spark DataFrame, repartition it to spread the data uniformly on each partition, make it persistent to avoid recalculation, each time the synopsis is needed and recast the date fields from strings back into timestamp format.

This methodology creates synopses with high accuracy regarding the number of rows, as was shown experimentally, with the divergence from the theoretical row count being less than **1%**.

- ## Data Cleaning

  Before delving into training the models it became evident that a few **Null** values existed in the dataset. Due to the sheer volume of data (25 years of data) we have chosen to replace **Null** values regarding the "District" column with the most frequent ones. This was achieved by identifying the most frequent "Beat" value for records with **Null** "District" entries, then using that "Beat" to determine the most common corresponding "District" number. In addition, we reduced the cardinality of crime types by keeping only those that had a total count above **4000**, while grouping all others into a single category named "Rare Crime".

- ## Preprocessing and Feature Selection

  The goal of the machine learning task was to create a District-level crime count predictor by "Primary Type" and "Date". To achieve this, the dataset was grouped by "Primary Type", "Year", "Month", "Day", "District" and the total crime count was computed for each group. To account for the skew in the count we applied the natural logarithmic transformation: $\boldsymbol{count \Rightarrow \ln(count + 1)}$, thus normalizing the distribution of values and improving performance. The features used for the prediction were the columns "Primary Type", "District", "Year", "Month", "Day". The values of "Primary Type" and "District" columns were indexed by a string indexer and later encoded by one hot encoding, to transform them into numerical format that the machine learning models could interpret, without introducing ordinal bias. The processed data were split into two sets, one for training and the other for testing purposes. The training set consists of all data up to and including the year **2020** and the test set consists of the data between the year **2021 and 2024**. Data from **2025** are excluded from both samples as they only cover January through April, thus providing too few values to be suitable.

- ## Regression Models, Synopses and Evaluation

  Three distinct models were used to create the crime count predictor:

  - **Gradient Boosted Trees Regressor** for its ability to handle non linearities in data, robustness to overfitting and high accuracy.
  - **Decision Tree Regressor** for its ability to handle non linearities in data and training speed.
  - **Linear Regression** for its simplicity, training speed and its use as a baseline model.

  The baseline model for the purposes of this project is the one were the training data stem from the entire dataset. Three synopses of various sizes were created using the reservoir sampling technique showcased before. The sizes were **40%, 30% and 20%**. In all the approaches, model performance was evaluated using **R^2, RMSE, MAE**, as well as the total training time, to facilitate a comprehensive comparison in terms of both accuracy of predictions and runtime.

Results

Correctness of Sampling Technique

Before examining training of the machine learning models it is important to showcase the correctness of the synopsis creation technique that was explained in the methodology section, through experimental results. To achieve this, we created a **20%** reservoir sample synopsis from the original dataset and ran 3 queries on both the original dataset and the synopsis to compare the results:

- **Query 1**: Days Ranked by Crime Percentage

```
Days Ranked By Crime Percentage (Original Dataset):
+---------------+----------------+
|Day of the week|Crime percentage|
+---------------+----------------+
|         Friday|            15.0|
|      Wednesday|           14.37|
|       Saturday|           14.34|
|        Tuesday|           14.28|
|       Thursday|           14.22|
|         Monday|           14.14|
|         Sunday|           13.65|
+---------------+----------------+

Days Ranked By Crime Percentage (Synopsis):
+---------------+----------------+
|Day of the week|Crime percentage|
+---------------+----------------+
|         Friday|           14.95|
|      Wednesday|           14.38|
|       Saturday|           14.31|
|        Tuesday|           14.29|
|       Thursday|           14.26|
|         Monday|           14.15|
|         Sunday|           13.65|
+---------------+----------------+

Query on original DataFrame took: 58.521920919418335 sec.
Query on synopsis DataFrame took: 2.503554582595825 sec.
```

Figure 2: Experimental Results of Query 1

- **Query 2**: Top 5 months with the most crimes

```
Top-5 months with most crimes (Original DF):
+-------+------+
|  Month|crimes|
+-------+------+
|   July|765501|
| August|757798|
|    May|728340|
|   June|727809|
|October|721938|
+-------+------+

Top-5 months with most crimes (Synopsis):
+-------+------+
|  Month|crimes|
+-------+------+
|   July|766580|
| August|756150|
|    May|731060|
|   June|728040|
|October|716550|
+-------+------+

Query on original DataFrame took: 47.7045795917511 sec.
Query on synopsis DataFrame took: 2.356328010559082 sec.
```

Figure 3: Experimental Results of Query 2

- **Query 3**: Arrest Percentages on domestic and non-domestic cases

```
Percentage of Arrests(Original Dataset)
Domestic Case: False || Arrest Percentage: 26.646%
Domestic Case: True || Arrest Percentage: 19.301%
================================================
Percentage of Arrests(Synopsis):
Domestic Case: False || Arrest Percentage: 26.646%
Domestic Case: True || Arrest Percentage: 19.301%


Query on original DataSet took: 85.26183915138245 sec.
Query on Synopsis took: 1.9232797622680664 sec.
```

Figure 4: Experimental Results of Query 3

Insights:

- Using an oversample rate of **1** (**no oversampling** on partition samples) we achieved a total row count, on the synopsis, of **1658835**, while the exact theoretical row count of the **20% synopsis** would be 8294210*20% = **1658842**. That is a relative difference of less than **1%**.
- As shown in the above figures the relative errors that occur are below **1%**, indicating that the sampling technique is able to preserve the statistical distribution of the original dataset. In addition, the performance of the queries is greatly improved, achieving a **44% faster** response time in the case of the third query. These results show that the sampling technique produces an efficient and correct synopsis.

Full Dataset Utilization Approach

The first approach and the baseline model for our comparison uses the entire dataset to train each one of the three models. Specifically, it uses all the data set entries starting from the year 2001 up to and including the year 2020. The results for each model are shown below:

Table 1: Performance results for full dataset utilization

Model	R^2	RMSE	MAE	Training Time
GBT Regression	0.74466	0.46578	0.35669	537.47 sec
Decision Tree Regression	0.71518	0.49193	0.37332	125.71 sec
Linear Regression	0.54991	0.61839	0.47781	157.36 sec

Synopses Training Approach

All the synopsis' models were tested for their predictive capabilities on the same testing set, consisting of data between 2021 and 2024, as the original full utilization approach.

40% Synopsis Training

The results for the 40% reservoir sampled synopsis models are summarized below:

Table 2: Performance results for 40% sample synopsis

Model	R^2	RMSE	MAE	Training Time
GBT Regression	0.53180	0.63072	0.51019	419.73 sec
Decision Tree Regression	0.46527	0.67404	0.54019	73.30 sec
Linear Regression	0.61463	0.57221	0.45345	38.69 sec

30% Synopsis Training

The results for the 30% reservoir sampled synopsis models are summarized below:

Table 3: Performance results for 30% sample synopsis

Model	R^2	RMSE	MAE	Training Time
GBT Regression	0.41788	0.70327	0.57607	362.77 sec
Decision Tree Regression	0.33359	0.75247	0.61281	66.33 sec
Linear Regression	0.63065	0.56019	0.45559	30.58 sec

20% Synopsis Training

The results for the 20% reservoir sampled synopsis models are summarized below:

Table 4: Performance results for 20% sample synopsis

Model	R^2	RMSE	MAE	Training Time
GBT Regression	0.13871	0.85545	0.70268	334.07 sec
Decision Tree Regression	0.06691	0.89039	0.73060	59.06 sec
Linear Regression	0.47517	0.66777	0.54999	24.89 sec

Insights:

- As expected, training on the **full dataset (years 2001-2020)** yields the best results regarding accuracy and errors. Notably:
    - **Gradient Boosted Trees Regression** yields the highest **R^2** value of **0.74466**, as well as the lowest **RMSE** and **MAE** errors, outperforming all other approaches and thus making it the most suitable choice when the main consideration is accuracy. However, it comes at a cost of longer training time at **537.47 sec**.
    - **Decision Tree Regression** results in similar, although slightly worse performance than GBT with a **0.71518 R^2** value and slight increase in error values. However, it trained significantly faster at **125.71 sec**, almost **4.3** times faster than GBT Regression with only **3.96 to 5.61%** performance loss. This shows that when the main consideration is speed, with good degree of accuracy then this model choice is more appropriate.
    - **Linear Regression** resulted in the weakest performance, showing relatively low **R^2** score, as well as higher **RMSE** and **MAE** errors, compared to the previous two models. Additionally, it was slower to train than the Decision Tree model, although still a being a lot faster than the GBT model (roughly **3.4** times faster). This indicates that linear regression struggled to capture the complexity of data, even after applying the natural log transformation to the target variable, normalizing the distribution.
- Regarding the **synopsis-based models** we can make the following observations:
    - The **training time** across all models and synopses reduces significantly with their size, especially on the **Linear Regression** and **Decision Tree** models.
    - The performance of **GBT** and **Decision Tree** steadily decline with the reduction in synopsis size, showing a direct dependence between the volume of data and the performance of these models.
    - **Linear Regression** is able to produce moderate to good levels of predictive accuracy, across all the synopsis approaches. This shows that under certain conditions this simpler model is more robust under data constraints.

- Especially on the **30% synopsis Linear Regression** produces an $\mathbf{R^2}$ score of **0.63065**, which is **15.3%** lower than the best performing **full dataset** GBT model. Unfortunately, the **RMSE** and **MAE** values are between **20.2% and 27.7%** higher. However, the training time here is astoundingly fast at **30.58 sec**, almost **17.6 times faster** than GBT Regression on the full dataset. This indicates that the **Linear Regression** model could be used as an alternative when speed, with a moderate degree of accuracy, over peak performance, is the main objective.

Concluding Remarks and Future Work

In this project, we have created a District-based Crime Count predictor by "Primary Type" and "Date", also known as a Spatiotemporal Crime Count Predictor. We proposed a method to create synopses of any size with distributed data using reservoir sampling and proved that it was able to produce synopses with high accuracy to the number of rows, while preserving the statistical distribution of the original dataset. Moreover, we evaluated the performance of three Regression models: Gradient Boosted Trees Regression (GBT), Decision Tree Regressor and Linear Regression and concluded that:

- In terms of prediction accuracy, the GBT model on the full dataset outperforms all other models, while
- In terms of training speed, the Decision Tree Regressor model on the full dataset is significantly faster than GBT, while suffering relatively low metric losses, making it the second-best model.

Finally, we studied the case of using synopses of various sizes, instead of the full dataset to train the ml models. Our experiments showed the direct dependence of more complex models, like GBT and Decision Tree, on data volume while also highlighting that simpler models, like Linear Regression, are more robust to data constraints and can produce satisfactory results, regarding accuracy of predictions, while having significantly faster training times. Thus, we can conclude that when predictive accuracy is critical, training a more complex model using the full dataset can provide the best results. However, when faster training with moderate to good predictive accuracy is preferred or when the full dataset cannot be used for reasons like time limits or large data volume, then training using a synopsis and a simpler model can procure the desired results.

For future work, the experiments in this project could be revisited by creating synopses using a different sampling technique that ensures better representation of all subgroups in the dataset, like stratified sampling. In addition, more experiments with the sizes of the synopses could further reveal the tradeoff between training speed and accuracy of predictions. Lastly instead of using classic regression models, experimentation using neural networks could procure even better results, regarding accuracy of predictions, as well as highlight the tradeoffs of full dataset utilization versus synopses, outside of the classic machine learning regression models.

References

[1] Big Data Processing and Analysis Lectures by Prof. Minos Garofalakis

[2] Classification and Regression – Spark 4.0.0 Documentation. Retrieved from: https://spark.apache.org/docs/latest/ml-classification-regression.html

[3] Apache Spark's PySpark Jupiter Notebooks by Prof. Nikolaos Giatrakos

[4] Chicago Data Portal (The csv dataset file was exported from here): https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/about_data